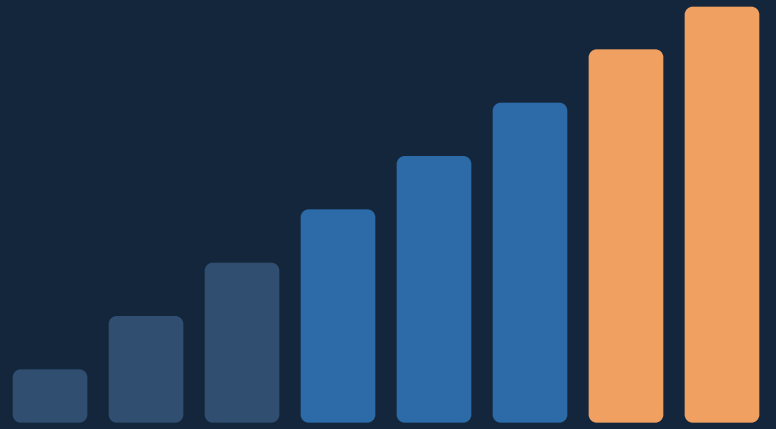


SUPREMETECH ENGINEERING

AI-Driven Development

A tool-agnostic level-up playbook, from first setup to spec-driven, test-first, agentic delivery

Examples use Claude Code, Codex and Kiro. Every tool is replaceable.

SupremeTech · 2026/10

WHY A LEVEL-UP PATH

AI amplifies the system it lands in

DORA's 2025 research with nearly 5,000 technology professionals found that the biggest returns come from investing in foundations, not from the tool itself.

Speed alone

Agents write code faster, but without guardrails the extra output turns into review load and rework.

Guardrails alone

Strict process without good context slows everyone down and pushes people back to manual work.

A level-up path

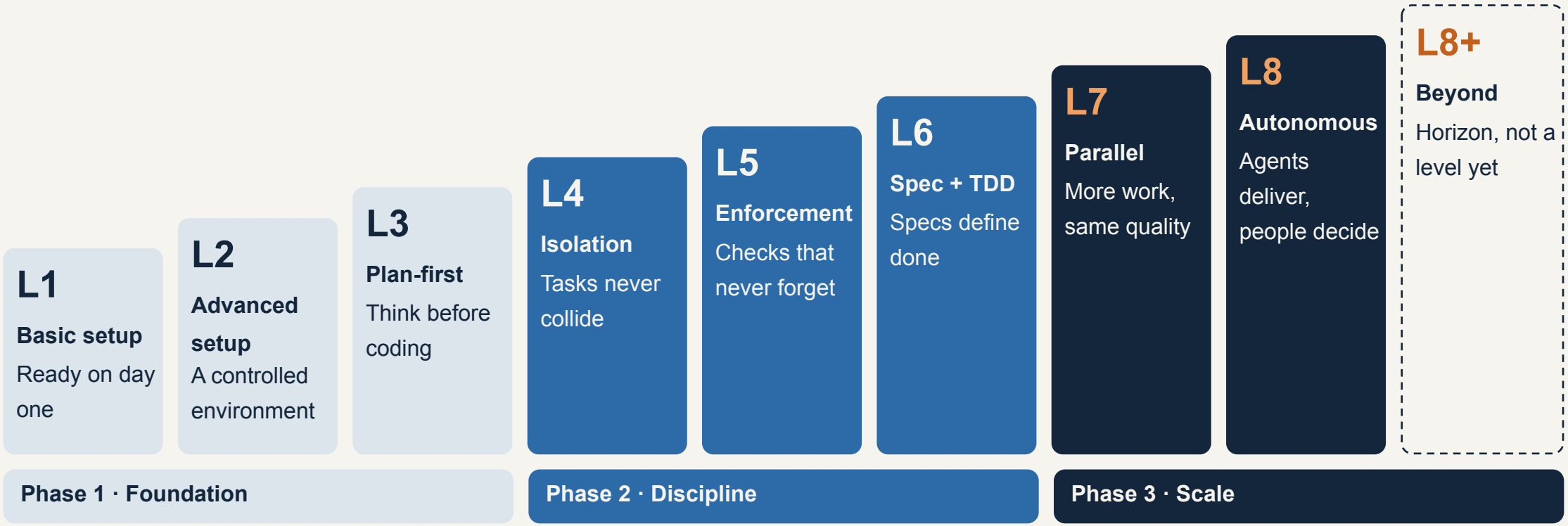
Each level adds one capability with its own speed and quality metrics, so the team earns autonomy step by step.

Source: Google Cloud, 2025 DORA AI Capabilities Model report

THE PATH AT A GLANCE

Eight levels in three phases

One capability per level, with its own speed and quality metrics. A tool can be used earlier, but it becomes required at its level.



TOOLS ARE REPLACEABLE

Practices stay fixed, tools plug in

Capability	Concept we own	Claude Code	Codex	Kiro
Agent instructions	One file, converted on a tool switch	CLAUDE.md	AGENTS.md	.kiro/steering/*.md
Agent init	Instructions, rules and skills	/init	/init	Generate Steering Docs
Scoped rules	Conventions per code area	.claude/rules/	AGENTS.md per directory	.kiro/steering/*.md
Reusable knowledge	Skills (Agent Skills format)	.claude/skills/	/skills	.kiro/skills/
Tool access	MCP, only when needed	.mcp.json	codex mcp	.kiro/settings/mcp.json
Plan before code	Plan approved in the PR	Plan mode	--sandbox read-only	.kiro/specs/ design
Enforcement	Four layers	Hooks	/hooks	.kiro/hooks/*.json
Isolation	Sandbox env + git worktree	claude -w	Any git worktree	Any git worktree
Headless in CI	CI runner + OIDC	claude-code-action@v1	codex exec	Kiro CLI headless mode

Vendor columns checked against vendor docs on 2026-10-08: code.claude.com, developers.openai.com/codex, kiro.dev

L1 · BASIC SETUP

Agent init: ready to use on day one

Speed metric and target

First agent-assisted PR for a new developer:
within 1 day [estimate].

Quality metric and target

Review comments about conventions:
under 1 per PR [estimate].

Rules to follow

- Run the tool's init command, then trim the instruction file by hand.
- Add about 3 scoped rules for the busiest code areas [estimate].
- Add about 3 skills for the most repeated tasks, such as a migration or a test [estimate].
- Use one approved agent per repo, and convert init files when switching tools.

Watch out for

- Init output repeats the README; keep only what the code cannot tell the agent.
- Rules that contradict the instruction file; keep each rule scoped to its area.
- Vague skill descriptions make the agent pick the wrong skill, or none.

Tool examples: /init, .claude/rules/, .claude/skills/ in Claude Code · /init, AGENTS.md per directory, skills in Codex · steering, skills in Kiro

L2 · ADVANCED SETUP

Sandbox, MCP and clean sessions

Speed metric and target

Time to first green build in the sandbox env:

under 1 hour [estimate].

Quality metric and target

Repeated agent mistakes that a rule already covers:

zero.

Rules to follow

- Every repo runs in a sandbox env on the CI image, with no production credentials.
- Add MCP only when a task needs outside data, with least access such as a read-only DB.
- One task per session; clear or compact the context before it fills.
- Turn repeated mistakes into rules and repeated procedures into skills; prune every sprint.

Watch out for

- A sandbox that drifts from the CI image, or secrets leaking in through .env or mounts.
- Data returned by MCP can carry prompt injection; treat it as data only.
- Long sessions lose early instructions, and patching a wrong direction tangles the diff.

Tool examples: Dev Containers, Codespaces, Context7 MCP, /clear, /compact and /rewind in Claude Code

L3 · PLAN-FIRST

Plan first, then write code

Speed metric and target

PRs that need a second design round:

under 10% [estimate].

Quality metric and target

Multi-file PRs with an approved plan attached:

100%.

Rules to follow

- Multi-file or unclear tasks start with a read-only plan; one-sentence diffs skip it.
- A plan names the files to change, the interfaces touched and how to verify.
- A person approves the plan before coding, and the plan stays with the PR.
- When the direction changes, update the plan first, then the code.

Watch out for

- Vague plans such as 'refactor module X' get approved without real review.
- Plans longer than one screen do not get read.
- Code drifts from the plan; reviewers compare the diff against it.

Tool examples: Claude Code plan mode, Codex --sandbox read-only, Kiro spec design, a second reviewing model

L4 · ISOLATION

One worktree per task, nothing shared

Speed metric and target

Time to set up a workspace for a new task:

under 2 minutes [estimate].

Quality metric and target

Incidents caused by state shared between tasks:

zero.

Rules to follow

- One task, one worktree, one branch, one session; never share a checkout.
- Create worktrees inside the sandbox env, which mounts the repo root.
- Each worktree gets its own ports, database schema, cache and storage.
- A setup script copies .env and installs dependencies from shared caches.

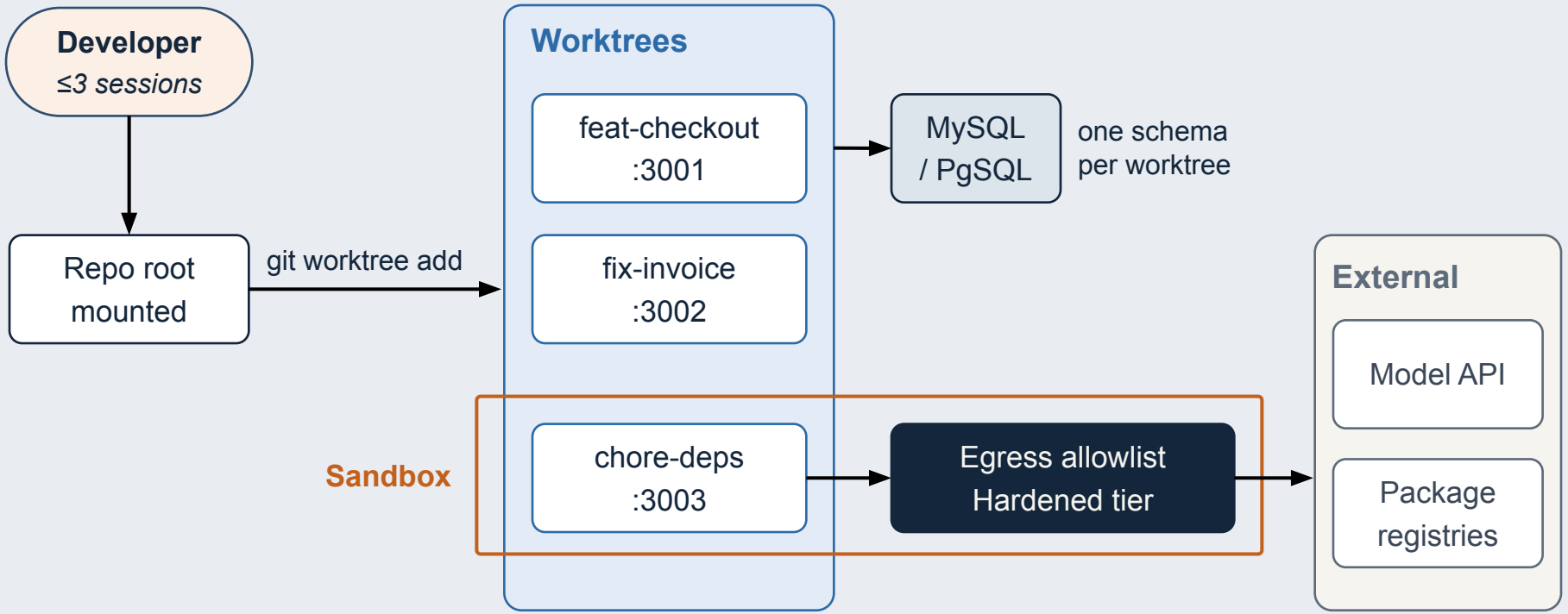
Watch out for

- Mounting only the worktree folder breaks its .git pointer.
- Hidden shared state, such as queues or Laravel storage/, makes tests flaky.
- Orphaned worktrees and branches pile up; prune them after every merge.

Tool examples: git worktree, claude -w, an internal setup script for ports, schemas and .env

L4 · TOPOLOGY

Git worktrees & Sandbox env



Mount the repo root, give each worktree its own ports and schema, copy .env in, and cap sessions per developer.

L5 · ENFORCEMENT

Four layers of checks

Speed metric and target

Mechanical comments in human review:

zero per PR.

Quality metric and target

Lint, type or secret findings reaching CI:

zero.

Rules to follow

- Run all four layers: agent hooks, git hooks, CI gate and merge policy.
- Agent hooks and git hooks give fast feedback; CI and merge policy decide.
- Destructive commands, secret reads and non-local migrations are blocked.
- Each PR passes a read-only reviewer agent before a human reviewer.

Watch out for

- Slow pre-commit pushes people to bypass it; keep it under 30 seconds [estimate].
- Agents pass checks by adding suppressions or loosening config; CI flags both.
- A noisy reviewer agent gets ignored; allow blocking comments only.

Tool examples: agent hooks, Lefthook, Prettier, ESLint, tsc, Pint, Larastan, gitleaks, Semgrep, branch rulesets

L6 · SPEC-DRIVEN + TDD

Specs and tests define done

Speed metric and target

QA round trips per feature:

at most 1 [estimate].

Quality metric and target

Criteria covered by tests:

100%, with mutation score above 70% [estimate].

Rules to follow

- Write each requirement in EARS, with a measurable acceptance criterion and an ID.
- Keep each spec to one reviewable PR; split it past about 7 criteria [estimate].
- Tests cite criterion IDs, come first, and must fail for the right reason.
- Change the spec before behavior; CI rejects test edits in implementation commits.

Watch out for

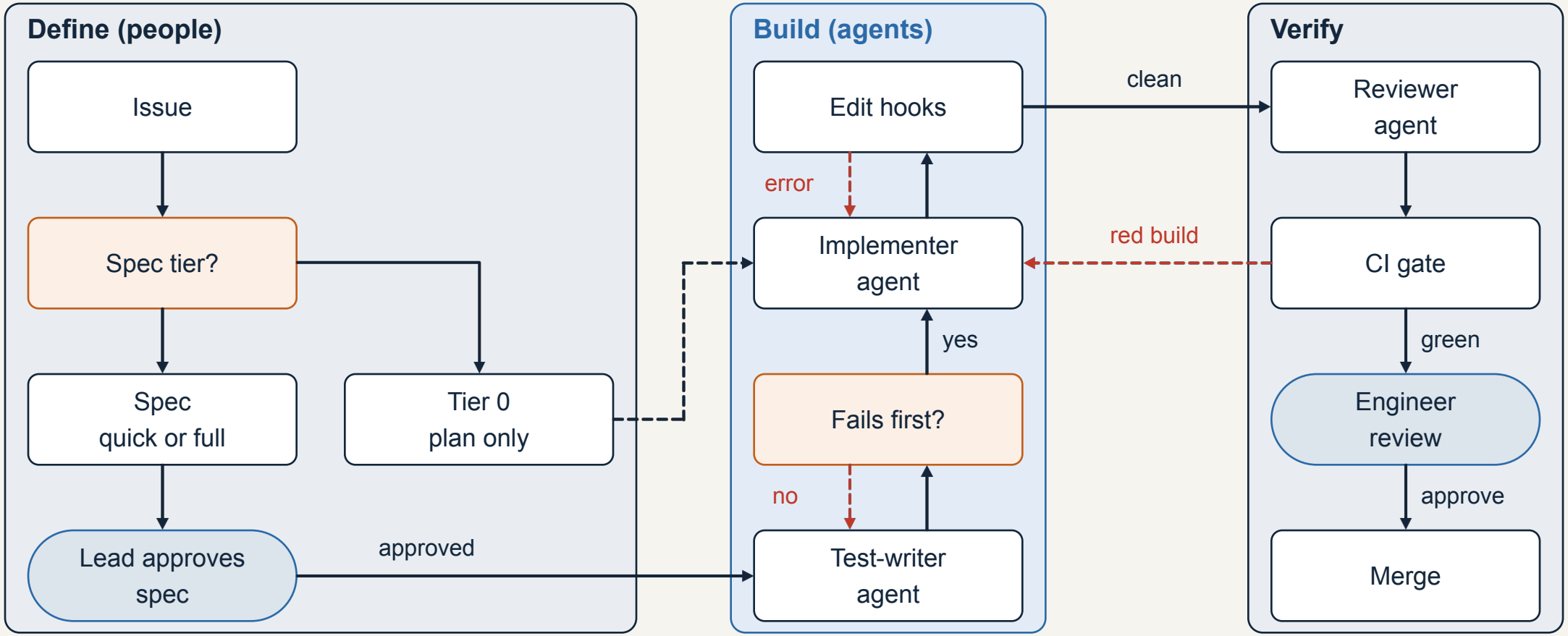
- Unclear specs with words like 'fast' or 'friendly' that no test can check.
- Missing out-of-scope and unchanged behavior, so the agent widens the scope.
- High coverage with weak asserts; mutation score shows the real strength.

Tool examples: Kiro specs, Spec Kit or an internal template, Jest, Pest or PHPUnit, Playwright, Stryker, Infection

L6 TO L8 · WORKFLOW

From issue to merge

Red dashed: work sent back



L7 · PARALLEL

Several sessions, one reviewer

Speed metric and target

Lead time from spec approval to merge:

30% below L6 [estimate].

Quality metric and target

Change failure rate:

no higher than at L6.

Rules to follow

- Run tasks in parallel only when each one has an approved spec.
- Keep at most 3 sessions per developer [estimate].
- Pick tasks that touch separate modules, so they rarely conflict.
- Review in batches instead of reacting to every session.

Watch out for

- Review becomes the bottleneck; lower the WIP limit when PR wait time grows.
- Tasks that touch the same files cause repeated merge conflicts.
- Constant context switching lowers the quality of review.

Tool examples: parallel sessions in Claude Desktop or any agent CLI, a WIP limit on the board

L8 · AUTONOMOUS AGENTIC

Agents deliver, people decide

Speed metric and target

Agent-ready issues that reach a PR unattended:
above 50% [estimate].

Quality metric and target

Agent PRs merged without manual rework:
above 70% [estimate].

Rules to follow

- Agents start only on issues labeled agent-ready by someone with rights.
- Unattended runs happen only in the hardened sandbox, on trusted repos.
- Agents cannot merge, and all four check layers stay on.
- Each run has a turn limit, a retry limit and a budget, with full logs.

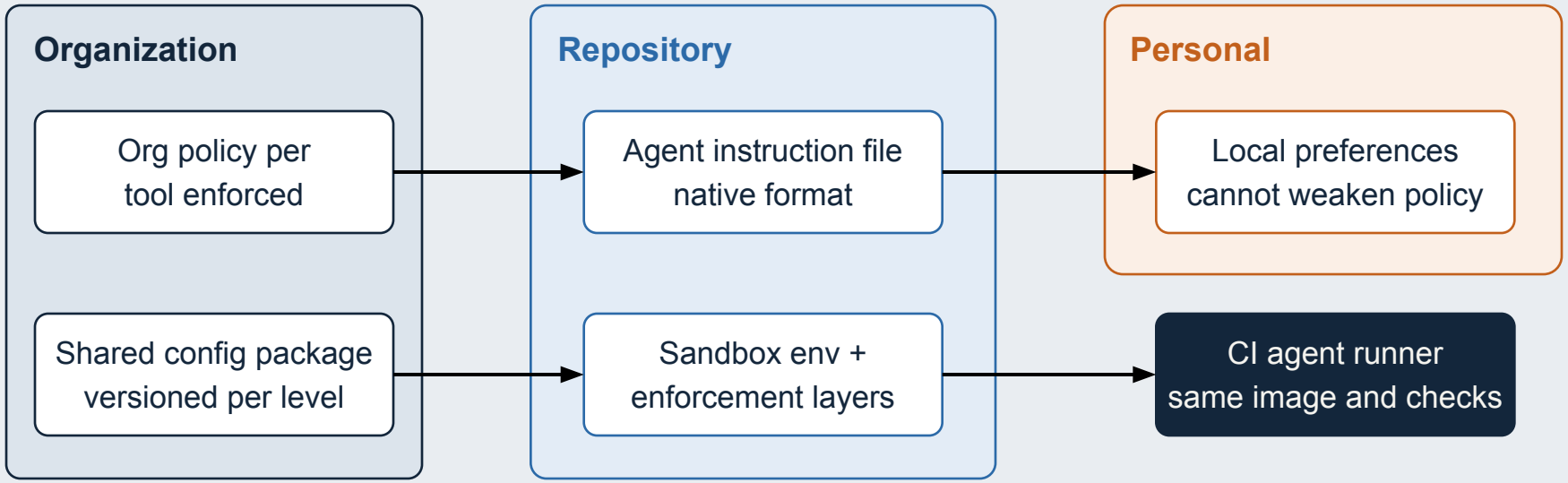
Watch out for

- Prompt injection through issues, comments or dependency docs.
- Agents open more low-quality PRs than people can review; cap open agent PRs.
- CI tokens with broad write rights; use OIDC with least privilege.

Tool examples: claude-code-action@v1, codex exec, Kiro CLI headless mode, GitHub OIDC, branch rulesets

STANDARDIZATION

Three layers keep every repo consistent



Moving up a level means enabling the next package version, whichever agent a team uses.

BEYOND L8 · HORIZON

Where the path goes next

These are directions, not levels yet. One becomes L9 once a team holds its L8 targets for three sprints [estimate].

Likely first

Production feedback loop

Agents read alerts and error tracking, write a bugfix spec with a failing test, and open a PR for people to review.
Needs: L6 bugfix specs and L8.

Risk: personal data in production signals, and prompt injection through logs.

Epic orchestration

A planner agent splits an epic into specs and tasks for other agents, and people approve the breakdown first.
Needs: L6 and a stable L8.

Risk: one wrong breakdown multiplies across every task.

Self-improving harness

A fixed eval suite scores rules, skills and instructions, and config changes ship only when scores hold.
Needs: the L2 feedback loop and the L5 checks.

Risk: overfitting to the evals, and constant config churn.

ROLLOUT

Proposed rollout

01

Pilot

[Pilot team] runs L1 to L3 on [one repo] for [__] sprints.
Exit: L1 to L3 metrics on target.

02

Expand

The pilot reaches L6 while [__] more teams start at L1.
Exit: L4 to L6 metrics on target for the pilot.

03

Scale

The platform team versions the shared package; teams move to L7 and L8.

Exit: L8 metrics on target in at least one repo.

Tool choice stays open per team, with one approved agent per repo. The sandbox env, the guardrails and the metrics stay the same.

Owner: [Name] · Review every [__] sprints · First check-in: [Date]

THANK YOU

Earn autonomy one gate at a time

Practices stay, tools change. Questions and feedback are very welcome.

FAQ · L1 BASIC SETUP

Start small, grow from evidence

Must the init set be perfect before we start?

Direction: begin with the minimum that makes the agent useful, and add only what evidence asks for.

Example: a trimmed instruction file, about 3 rules and 3 skills [estimate], with L2 adding more after real mistakes.

Who owns the instruction file, rules and skills?

Direction: one accountable owner per repo, and changes go through the same review as code.

Example: the repo tech lead approves every change by PR, and each rule states the mistake it prevents.

What if people prefer different agents?

Direction: keep one source of agent setup per repo, so conventions never split.

Example: one approved agent per repo at a time, with a one-time conversion of init files when switching.

Signal to watch: review comments about conventions, target under 1 per PR [estimate].

FAQ · L2 ADVANCED SETUP

Agent inside, credentials outside

What runs in the sandbox, and what stays on the host?

Direction: whatever the agent executes runs inside; whatever holds personal or production credentials stays outside.

Example: agent, builds, tests, dev servers and databases inside; IDE, browser and SSH keys on the host, with a forwarded SSH agent.

When is MCP worth adding?

Direction: add outside access only when the agent keeps needing it, with the least privilege that works.

Example: a read-only schema or ticket server, removed again if nobody used it in a sprint.

How do we know context management works?

Direction: a lesson learned once should not need to be learned again.

Example: track repeated mistakes that a rule already covers; if any appear, rewrite the rule or shorten sessions.

Signal to watch: repeated mistakes that a rule already covers, target zero.

FAQ · L3 PLAN-FIRST

Plan what matters, on one screen

Doesn't planning slow down small tasks?

Direction: spend planning effort in proportion to the risk of going the wrong way.
Example: skip the plan when the diff fits in one sentence, and keep every other plan to one screen.

What makes a plan ready to approve?

Direction: a reviewer must be able to predict the change before it is made.
Example: the plan lists the files to change, the interfaces touched and how the result is verified.

What if the plan turns out wrong mid-way?

Direction: a change of direction must be visible to the reviewer before code follows it.
Example: stop, update the plan and get it approved again, then continue instead of patching forward.

Signal to watch: PRs that need a second design round, target under 10% [estimate].

FAQ · L4 ISOLATION

Own worktree, nothing shared

Can we use worktrees before L4?

Direction: adopt a practice as soon as it helps; a level only marks when it becomes required.

Example: teams use worktrees from L1, and L4 adds the rules for ports, schemas, caches and storage.

Do worktrees cost too much disk and setup time?

Direction: share what is immutable, isolate what is mutable, and automate the setup.

Example: pnpm's shared store and a shared Composer cache, a setup script for .env, and pruning after merge

What still leaks between worktrees?

Direction: any state that lives outside git needs its own isolation rule.

Example: prefix queues and Redis keys, and give each worktree its own storage/ folder and browser profile.

Signal to watch: flaky tests that fail only in parallel runs, target zero.

FAQ · L5 ENFORCEMENT

Checks outside the agent, no bypass

What if a check blocks an urgent hotfix?

Direction: make the urgent path explicit and small, never a silent bypass.

Example: a documented hotfix lane with a minimal CI gate, and a follow-up PR within one day [estimate].

What if the agent games the checks?

Direction: treat any weakening of a check as a change that needs approval.

Example: CI fails on new eslint-disable or @ts-ignore, a larger Larastan baseline, or unapproved lint config edits.

Isn't a reviewer agent just more noise?

Direction: automated review earns attention only when nearly every comment matters.

Example: blocking issues only, each with a reason, and drop rules that humans keep overruling.

Signal to watch: new suppressions or config changes merged without approval, target zero.

FAQ · L6 SPEC + TDD

Clear, small, testable specs

Who writes specs, and isn't it extra work?

Direction: the person accountable for the outcome owns the spec; the agent does the typing.

Example: the feature owner edits an agent draft made from the issue; a quick spec is a few EARS lines.

How small is small enough?

Direction: a spec should produce one change a person can review in one sitting.

Example: about 7 acceptance criteria at most [estimate], one bounded context, split by user-visible outcome.

How do we know the tests are good?

Direction: a good test fails when the behavior is wrong, not just when code is missing.

Example: tests cite criterion IDs, fail before code exists, and are judged by mutation score.

Signal to watch: QA round trips per feature, target at most 1 [estimate].

FAQ · L7 PARALLEL

Control at gates, not by watching

How does one reviewer stay in control?

Direction: concentrate human attention at a few gates, and make the agent stop when it leaves the agreed scope.

Example: review the plan and the PR only, with a hook that stops edits outside the plan, to tests or to migrations.

How do we see every session at once?

Direction: the system should call the reviewer, not the other way around.

Example: notification and stop hooks ping Slack, and draft PRs opened on day one act as the status board.

How many sessions should one developer run?

Direction: let review capacity, not agent capacity, set the number.

Example: start with 2, stay at 3 or fewer [estimate], and lower it as soon as PR wait time grows.

Signal to watch: how long sessions wait for a person, and plan deviations that no hook caught (target zero).

FAQ · L8 AUTONOMOUS (1 OF 2)

Review specs and evidence, not every line

Shift review to the spec

Direction: put human judgment where it is cheapest: before code exists.

Example: approve short specs first; code review then checks evidence against them.

Review depth follows risk

Direction: spend reading time in proportion to the damage a mistake could do.

Example: full review for payments, auth and migrations; tests and summary for medium; sampling for low.

Evidence before diff

Direction: verify behavior directly instead of inferring it from code.

Example: acceptance traces and a preview link, checked by QA or the product owner.

Machines guard structure

Direction: rules a machine can check should never cost human review time.

Example: dependency-cruiser and deptrac fail CI when code crosses module boundaries.

Example budget: 10 features: 2 high at 30 min, 5 medium at 10 min, 3 low at 3 min, about 2 hours a day [estimate].

FAQ · L8 AUTONOMOUS (2 OF 2)

One feature map a day, riskiest first

Direction: review by feature, riskiest first.

Example: a daily map compiled by an agent from PRs, tests and specs (example data).

Feature (spec)	Criteria met	Risk	Evidence	Deviations
FEAT-142 Checkout coupon	6 of 6	High: payments	Trace, preview link	1 file outside plan
FEAT-151 Invoice PDF fix	3 of 3	Medium	Failing test now green	None
CHORE-88 Upgrade Pint	n/a	Low	CI green	None

Flags split merge from release

Direction: decouple merging code from exposing it to users.

Example: features merge behind a flag, turned on after a behavior check.

Cap at review capacity

Direction: never produce faster than the team can review.

Example: agents finish no more features a day than the team reviewed last week.

Weekly drift review

Direction: look at the whole regularly to catch what daily reviews miss.

Example: read the combined diff by module once a week.